Account-read outages: operator manual

An account-read failure is a warning, never a sell signal (AK, 2026-10-09). The managed supervisor keeps the native SL/TS orders at the venue and pauses local supervision. It records one warning, keeps reading on every poll, and resumes the same authorized protection once a fresh, consistent readback arrives. This manual covers the operator commands only. The policy itself is described in read outages and recovery limits.

The screenshots below are actual output from `kis_hl.cli`. They were produced against a temporary SQLite database driven by the offline scripted supervisor in `scripts/smoke_read_outage.py`, using synthetic timestamps. No network, credentials, signed action or `--live` flag was used, and no live outage was exercised.

1. Deliver the outage warning (S3)

Hermes runs the listing on its own schedule. It delivers each `message`, then reports the outcome. Listing is read-only. A `failed` result leaves the row pending and records the attempt, so the next run retries it. Notification handling never changes positions, exits or orders.

```
python -m kis_hl.cli supervisor notifications --venue hyperliquid
python -m kis_hl.cli supervisor notification-result --venue hyperliquid --notification-id ID --result failed
python -m kis_hl.cli supervisor notification-result --venue hyperliquid --notification-id ID --result delivered
```

```
$ python -m kis_hl.cli supervisor notifications --venue hyperliquid
{
  "notifications": [
    {
      "created_ms": 1791800001000,
      "delivery_attempts": 0,
      "episode_ms": 1791800001000,
      "id": 1,
      "kind": "read_outage",
      "last_attempt_ms": null,
      "message": "WARNING hl:BTC (hyperliquid, owner 8a849f3d): account reads unavailable since 2026-10-12T10:13:21+00:00
(account_snapshot, phase unknown, HTTP 502). Warning only: no exit requested. Native orders retained:
0x2d76eaa59f294a88abefda201f6a6983. Local supervision paused (local protections: local_trailing). Last successful observation:
2026-10-12T10:13:20+00:00.",
      "mode": "live",
      "payload": {
        "instrument": "hl:BTC",
        "local_protections": [
          "local_trailing"
        ],
        "local_supervision": "paused",
        "native_orders_retained": [
          "0x2d76eaa59f294a88abefda201f6a6983"
        ],
        "read_health": {
          "consecutive_failures": 1,
          "error": {
            "category": "TransientInfoError",
            "http_status": 502,
            "kind": "http"
          },
          "last_failure_ms": 1791800001000,
          "last_success_ms": 1791800000030,
          "operation": "account_snapshot",
          "phase": null,
          "since_ms": 1791800001000,
          "status": "unavailable"
        },
        "reason": "Account snapshot unavailable (TransientInfoError); consecutive failures=1; warning only; no exit requested;
native orders retained, local supervision paused",
        "state": "DEGRADED",
        "supervision_reason": "Account reads unavailable: local SL/TS, exits and fill reconciliation cannot run; native orders
are retained but not re-verified",
        "venue": "hyperliquid"
      },
      "position_id": "8a849f3da8f64613ab5727972f4d2c8d"
    }
  ]
}
[exit 0]

$ python -m kis_hl.cli supervisor notification-result --venue hyperliquid --notification-id 1 --result failed
{
  "delivered_ms": null,
  "delivery_attempts": 1,
  "id": 1,
  "kind": "read_outage",
  "last_attempt_ms": 1791886865878
}
[exit 0]

$ python -m kis_hl.cli supervisor notification-result --venue hyperliquid --notification-id 1 --result delivered
{
  "delivered_ms": 1791886866008,
  "delivery_attempts": 2,
  "id": 1,
  "kind": "read_outage",
  "last_attempt_ms": 1791886866008
}
[exit 0]
```

Each owner gets exactly one `read_outage` row per episode, however many polls fail and even across restarts. Messages and payloads omit the account scope and any transport text. They report only the allowlisted error class, kind and HTTP status.

2. Read the degraded owner during the outage (S1, S4)

```
python -m kis_hl.cli order status --id POSITION_ID
```

```
$ python -m kis_hl.cli order status --id 8a849f3da8f64613ab5727972f4d2c8d
{
  "state": "DEGRADED",
  "reason": "Account snapshot unavailable (TransientInfoError); consecutive failures=69; warning only; no exit requested;
native orders retained, local supervision paused",
  "exit_requested_ms": null,
  "diagnostics": {
    "read_health": {
      "consecutive_failures": 69,
      "error": {
        "category": "TransientInfoError",
        "http_status": 502,
        "kind": "http"
      },
      "last_failure_ms": 1791800069000,
      "last_success_ms": 1791800000030,
      "operation": "account_snapshot",
      "phase": null,
      "since_ms": 1791800001000,
      "status": "unavailable"
    },
    "supervision": {
      "local_protections": [
        "local_trailing"
      ],
      "local_supervision": "paused",
      "native_orders_retained": [
        "0x2d76eaa59f294a88abefda201f6a6983"
      ],
      "supervision_reason": "Account reads unavailable: local SL/TS, exits and fill reconciliation cannot run; native orders
are retained but not re-verified"
    },
    "exit": {
      "attempts_exhausted": false,
      "deadline_at_ms": null,
      "deadline_exhausted": false,
      "exit_attempts": 0,
      "max_exit_attempts": 3,
      "provenance": null,
      "requested_ms": null,
      "withdrawn": []
    }
  }
}
[exit 0]
```

Check four things:

- `exit_requested_ms` stays `null`;
- `read_health` names the failing operation, phase and classification;
- `supervision.native_orders_retained` lists the owned venue orders, which stay in place but cannot be re-verified while reads fail;
- `local_supervision` is `paused`.

While reads fail, local SL/TS cannot act. That is the accepted risk of the warning-only policy.

3. Confirm recovery (S1, S3)

On the first fresh, consistent readback the owner returns to its authorized protection without new SL/TS orders, and one `read_recovered` notice is recorded.

```
$ python -m kis_hl.cli supervisor notifications --venue hyperliquid
[
  {
    "id": 2,
    "kind": "read_recovered",
    "message": "RECOVERED hl:BTC (hyperliquid, owner 8a849f3d): account reads recovered at 2026-10-12T10:14:30+00:00 after 69
failed reads since 2026-10-12T10:13:21+00:00. State PROTECTED: Coverage verified; local trailing active. Local supervision
active."
  }
]
[exit 0]
```

If the notice reports `INTERVENTION`, the reads recovered but reconciliation found an identity, ownership or evidence problem. That problem still needs manual reconciliation.

4. Resolve a legacy read-outage latch (S5)

Code before issue 59 could latch an exit during a read outage. Once its deadline passed, the owner stayed in `INTERVENTION`. `order recover` now refuses such an owner and explains why. Every step after that requires separate, explicit authorization for that exact owner.

```
$ python -m kis_hl.cli order recover --id 8a849f3da8f64613ab5727972f4d2c8d
{
  "error": "Recovery refused. Exit deadline budget exhausted: the exit request at 1791800080000 ms had exit_deadline_ms=60000
and expired at 1791800140000 ms. Recovery keeps budgets unchanged, so the supervisor would return to INTERVENTION immediately.
Exit provenance: read_outage_latch (event). Only under separate explicit authorization, withdraw exactly this request with
`order withdraw-read-exit --id 8a849f3da8f64613ab5727972f4d2c8d --exit-requested-ms 1791800080000` and then run `order
recover`."
}
[exit 1]

$ python -m kis_hl.cli order status --id 8a849f3da8f64613ab5727972f4d2c8d
{
  "intervention": {
    "current_blocker": {
      "reason": "Exit attempt or deadline budget exhausted",
      "since_ms": 1791800141000
    },
    "first_cause": null
  },
  "exit": {
    "requested_ms": 1791800080000,
    "deadline_exhausted": true,
    "exit_attempts": 0,
    "provenance": {
      "evidence": "event",
      "kind": "read_outage_latch",
      "note": "The INTERVENTION transition persisted at exit_requested_ms carries the read-outage reason"
    }
  }
}
[exit 0]

$ python -m kis_hl.cli order withdraw-read-exit --id 8a849f3da8f64613ab5727972f4d2c8d --exit-requested-ms 1791800080000
{
  "state": "INTERVENTION",
  "reason": "Read-outage exit request withdrawn by explicit command; run order recover to resume supervision",
  "exit_requested_ms": null,
  "withdrawn_exit_requests": [
    {
      "exit_requested_ms": 1791800080000,
      "provenance": {
        "evidence": "event",
        "kind": "read_outage_latch",
        "note": "The INTERVENTION transition persisted at exit_requested_ms carries the read-outage reason"
      },
      "withdrawn_ms": 1791886866239
    }
  ]
}
[exit 0]

$ python -m kis_hl.cli order recover --id 8a849f3da8f64613ab5727972f4d2c8d
{
  "state": "ENTERING",
  "reason": "Explicit recovery requested; budgets and ownership unchanged"
}
[exit 0]

# next supervisor tick -> PROTECTED; orders sent so far: ['entry', 'stop']
```

1. Run `order status --id ID`. Confirm that `exit.provenance.kind` is `read_outage_latch`, that `exit_attempts` is 0, and that `intervention.current_blocker` holds the deadline blocker.
2. Run `order withdraw-read-exit --id ID --exit-requested-ms EXACT_MS`. The command refuses a wrong time, any other provenance, an existing exit attempt, an UNKNOWN or unresolved entry/add, a recorded protective fill, or a pending entry cancellation. A flag on an older independent request shows provenance `ambiguous_legacy`, cannot be withdrawn, and that earlier request is honored. It never renews a budget and never sends an order.

3. Run `order recover --id ID`. The next supervisor tick reconciles with unchanged budgets. In the example it returns to `PROTECTED` with no new orders.

If any check fails, manage the residual position under a separately authorized procedure. Never edit SQLite.